

Scalable REST API with Auth & Role-Based Access

Backend Developer Intern — Complete Assignment Solution

Stack: FastAPI · PostgreSQL · Redis · Docker Compose

Auth: JWT (python-jose) + bcrypt password hashing (passlib)

RBAC: user role (own tasks) vs admin role (all tasks + user management)

Frontend: Vanilla JS + CSS — no frameworks, zero build step, served via nginx

API Versioning: All routes under /api/v1/ prefix; easy to add /api/v2/ later

Security: Input validation (Pydantic v2), soft delete, password strength checks

Vikas · vikas2227031583@gmail.com · github.com/Vikas84214

To,
Sonika,
Primetrade.ai Hiring Team

SUBJECT: Submission — Backend Developer Intern Assignment

Please find my assignment attached herewith for your review.

I have built a complete Scalable REST API with JWT Authentication and Role-Based Access Control, along with a basic frontend UI that connects to all API endpoints. The system is fully containerised and starts with a single docker compose up command.

What has been implemented:

• User Registration & Login	JWT auth with bcrypt password hashing; password strength validation
• Role-Based Access (RBAC)	user role (own tasks only) vs admin role (all tasks + user management)
• CRUD for Tasks	Create, Read, Update (PATCH), soft-delete with pagination & filters
• API Versioning	All routes under /api/v1/; designed for easy /api/v2/ addition
• Error Handling	Global validation error handler; HTTP 400/401/403/404/409/422 responses
• Input Validation	Pydantic v2 field validators — password strength, non-empty title, enum checks
• Swagger / ReDoc	Auto-generated at /docs and /redoc with full request/response schemas
• Database Schema	PostgreSQL with SQLAlchemy ORM — users + tasks tables with FK constraints
• Frontend UI	Vanilla JS: register, login, JWT-protected dashboard, CRUD task UI
• Docker Compose	4 services: PostgreSQL, Redis, FastAPI backend, nginx frontend
• Security	HTTPBearer JWT, soft delete (no data loss), CORS middleware, no SQL injection
• Scalability Note	README covers Redis caching, Kubernetes, load balancing, read replicas

Looking forward to your feedback.

Yours sincerely,

Vikas

vikas2227031583@gmail.com · GitHub: <https://github.com/Vikas84214>

Table of Contents

1. Architecture & System Design
2. Project Structure
3. Database Schema (models.py — full file)
4. Configuration (config.py — full file)
5. Database Setup (database.py — full file)
6. Security Utilities (core/security.py — full file)
7. Auth Dependency (core/deps.py — full file)
8. Pydantic Schemas — User (schemas/user.py — full file)
9. Pydantic Schemas — Task (schemas/task.py — full file)
10. CRUD — User (crud/user.py — full file)
11. CRUD — Task (crud/task.py — full file)
12. Auth Endpoints (api/v1/endpoints/auth.py — full file)
13. User Endpoints (api/v1/endpoints/users.py — full file)
14. Task Endpoints (api/v1/endpoints/tasks.py — full file)
15. API Router (api/v1/router.py — full file)
16. FastAPI App Entry Point (main.py — full file)
17. requirements.txt & Dockerfile (full files)
18. docker-compose.yml (full file)
19. Frontend — index.html (full file)
20. Frontend — style.css (full file)
21. Frontend — app.js (full file)
22. README.md (full file)
23. Example curl Commands & API Response Shapes
24. Role-Based Access Matrix
25. Scalability & Production Readiness Notes

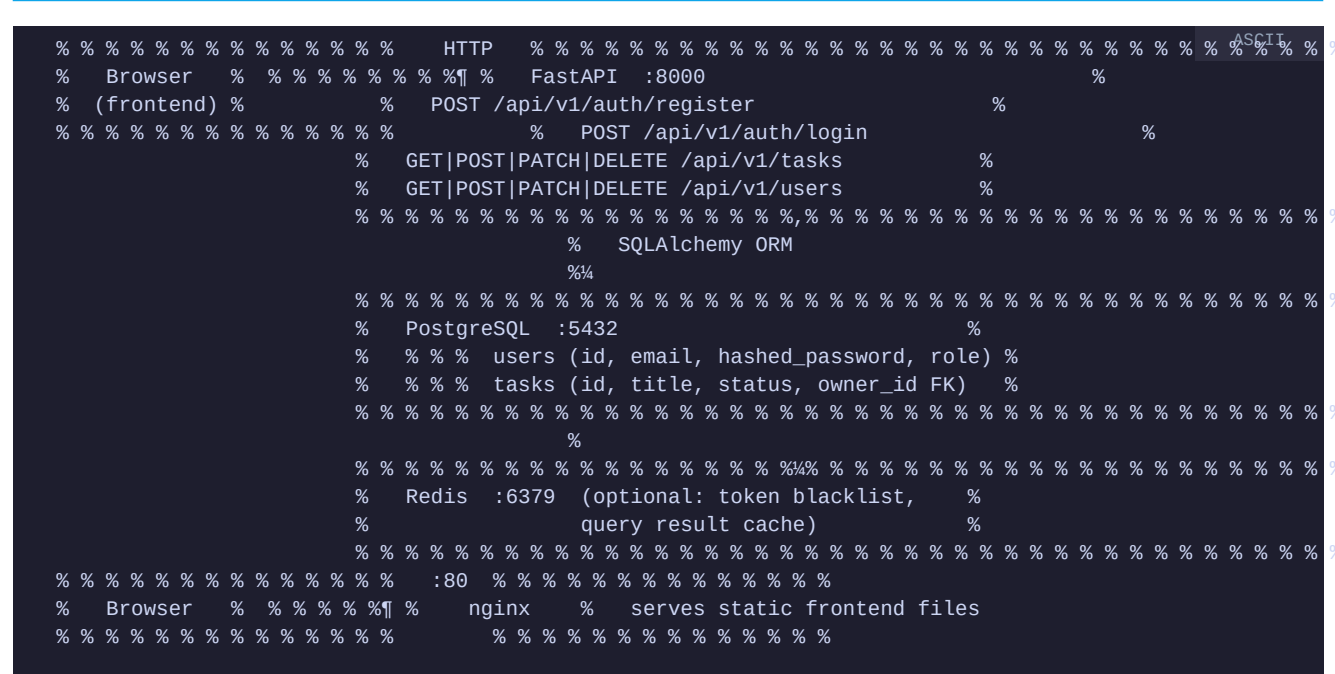
1 · Architecture & System Design

The system follows a layered architecture: HTTP layer (FastAPI routes) !' Business layer (CRUD functions) !' Data layer (SQLAlchemy ORM + PostgreSQL). JWT tokens are stateless — the worker validates them on every request without a database round-trip. Role-based access is enforced in the dependency layer (`core/deps.py`), keeping route handlers clean.

Request Lifecycle — Authenticated CRUD

- 1. Client sends HTTP request with Authorization: Bearer <JWT> header
- 2. FastAPI HTTPBearer extracts the token string
- 3. `get_current_user()` dependency decodes & verifies the JWT (python-jose)
- 4. User is fetched from PostgreSQL and confirmed `is_active=True`
- 5. Role check: `get_current_admin()` raises 403 if `role != 'admin'`
- 6. Route handler calls CRUD function (e.g. `get_tasks`, `create_task`)
- 7. CRUD function queries PostgreSQL via SQLAlchemy ORM
- 8. Result is serialised by Pydantic schema and returned as JSON

Architecture Diagram



JWT Authentication Flow

```
Register: POST /api/v1/auth/register
    !' validate input (Pydantic) !' check email unique !' bcrypt hash password
    !' INSERT user row !' return UserOut (no password)

Login: POST /api/v1/auth/login
    !' fetch user by email !' bcrypt verify password
    !' create JWT: {"sub": user_id, "role": user_role, "exp": now+60min}
    !' return {access_token, token_type: "bearer", user}

Protected route: GET /api/v1/tasks
    Authorization: Bearer eyJhbGci...
    !' HTTPBearer extracts token
    !' jose.jwt.decode() verifies signature & expiry
    !' SELECT user WHERE id=sub AND is_active=true
    !' inject User object into route handler
```

2 · Project Structure

TEXT

```
task-manager/
% % % docker-compose.yml      # Section 18 – 4 services: db, redis, backend, frontend
% % % .env.example           # Environment variable template
% % % README.md              # Section 22 – setup + curl examples
%
% % % backend/
%   % % % Dockerfile         # Section 17 – Python 3.11 slim image
%   % % % requirements.txt    # Section 17 – pinned dependencies
%   % % % app/
%     % % % main.py           # Section 16 – FastAPI app, CORS, error handlers
%     % % % config.py         # Section 4 – pydantic-settings
%     % % % database.py       # Section 5 – SQLAlchemy engine + session
%     % % % models.py         # Section 3 – ORM: User, Task (with enums)
%     % % % schemas/
%       % % % user.py         # Section 8 – UserRegister, UserLogin, TokenResponse
%       % % % task.py         # Section 9 – TaskCreate, TaskUpdate, TaskListResponse
%     % % % core/
%       % % % security.py     # Section 6 – JWT encode/decode, bcrypt hash/verify
%       % % % deps.py         # Section 7 – get_current_user, get_current_admin
%       % % % crud/
%         % % % user.py       # Section 10 – DB operations for users
%         % % % task.py       # Section 11 – DB operations for tasks
%       % % % api/v1/
%         % % % router.py     # Section 15 – combines all endpoint routers
%         % % % endpoints/
%           % % % auth.py     # Section 12 – /auth/register, /auth/login
%           % % % users.py    # Section 13 – /users CRUD
%           % % % tasks.py    # Section 14 – /tasks CRUD
%
% % % frontend/
%   % % % index.html          # Section 19 – single-page app markup
%   % % % style.css           # Section 20 – responsive styles
%   % % % app.js              # Section 21 – API calls, auth, task management
```

3 · Database Schema (app/models.py)

Two tables with a one-to-many relationship (one user ! many tasks). Enums enforce valid values at the database level. Soft-delete (is_deleted) preserves data history — tasks are never physically removed.

Schema Diagram

[illegible]

```
from datetime import datetime
from sqlalchemy import Column, Integer, String, Boolean, DateTime, Text, ForeignKey, Enum
from sqlalchemy.orm import relationship
import enum

from app.database import Base

class UserRole(str, enum.Enum):
    user = "user"
    admin = "admin"

class TaskStatus(str, enum.Enum):
    todo = "todo"
    in_progress = "in_progress"
    done = "done"

class TaskPriority(str, enum.Enum):
    low = "low"
    medium = "medium"
    high = "high"

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
```

```
full_name = Column(String(100), nullable=False)
email = Column(String(255), unique=True, index=True, nullable=False)
hashed_password = Column(String(255), nullable=False)
role = Column(Enum(UserRole), default=UserRole.user, nullable=False)
is_active = Column(Boolean, default=True)
created_at = Column(DateTime, default=datetime.utcnow)
updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

tasks = relationship("Task", back_populates="owner", cascade="all, delete-orphan")
```

```
class Task(Base):
    __tablename__ = "tasks"

    id = Column(Integer, primary_key=True, index=True)
    title = Column(String(200), nullable=False)
    description = Column(Text, nullable=True)
    status = Column(Enum(TaskStatus), default=TaskStatus.todo, nullable=False)
    priority = Column(Enum(TaskPriority), default=TaskPriority.medium, nullable=False)
    due_date = Column(DateTime, nullable=True)
    is_deleted = Column(Boolean, default=False) # soft delete
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    owner_id = Column(Integer, ForeignKey("users.id"), nullable=False)
    owner = relationship("User", back_populates="tasks")
```

4 · Configuration (app/config.py)

pydantic-settings reads env vars automatically. Change SECRET_KEY in production to a 64-character random string. ACCESS_TOKEN_EXPIRE_MINUTES controls session duration.

```
from pydantic_settings import BaseSettings
from datetime import timedelta

class Settings(BaseSettings):
    APP_NAME: str = "TaskManager API"
    API_V1_PREFIX: str = "/api/v1"
    DEBUG: bool = False

    # Database
    DATABASE_URL: str = "postgresql://appuser:apppass@db:5432/taskdb"

    # JWT
    SECRET_KEY: str = "change-this-to-a-long-random-secret-in-production"
    ALGORITHM: str = "HS256"
    ACCESS_TOKEN_EXPIRE_MINUTES: int = 60

    # Redis (optional caching)
    REDIS_URL: str = "redis://redis:6379/0"

    class Config:
        env_file = ".env"

settings = Settings()
```

PYTHON

5 · Database Setup (app/database.py)

Connection pool size=10, overflow=20. pool_pre_ping=True prevents stale connection errors after PostgreSQL restarts. get_db() is a FastAPI dependency that guarantees session cleanup via try/finally.

```
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker

from app.config import settings

engine = create_engine(
    settings.DATABASE_URL,
    pool_size=10,
    max_overflow=20,
    pool_pre_ping=True,
)

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

class Base(DeclarativeBase):
    pass

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

def init_db():
    from app import models # noqa
    Base.metadata.create_all(bind=engine)
```

PYTHON

6 · Security Utilities (app/core/security.py)

Three security functions: `hash_password()` uses `bcrypt` with auto-generated salt. `verify_password()` does constant-time comparison (no timing attacks). `create_access_token()` embeds `user_id` and `role` in JWT payload; `decode_token()` verifies signature and expiry in one call.

```
from datetime import datetime, timedelta
from typing import Optional

from jose import JWTError, jwt
from passlib.context import CryptContext

from app.config import settings

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")


def hash_password(password: str) -> str:
    return pwd_context.hash(password)


def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)


def create_access_token(data: dict, expires_delta: Optional[timedelta] = None) -> str:
    to_encode = data.copy()
    expire = datetime.utcnow() + (
        expires_delta or timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)
    )
    to_encode.update({"exp": expire, "iat": datetime.utcnow()})
    return jwt.encode(to_encode, settings.SECRET_KEY, algorithm=settings.ALGORITHM)


def decode_token(token: str) -> dict:
    """
    Decode and verify a JWT. Raises JWTError on invalid/expired tokens.
    """
    return jwt.decode(token, settings.SECRET_KEY, algorithms=[settings.ALGORITHM])
```

7 · Auth Dependency (app/core/deps.py)

`get_current_user()` is the base dependency injected into every protected route. It extracts the Bearer token, decodes the JWT, and fetches the live user from DB. `get_current_admin()` wraps it and adds the role check — raising 403 for non-admins.

```
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from jose import JWTError
from sqlalchemy.orm import Session

from app.core.security import decode_token
from app.database import get_db
from app.models import User

bearer_scheme = HTTPBearer()

def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(bearer_scheme),
    db: Session = Depends(get_db),
) -> User:
    token = credentials.credentials
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate credentials.",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = decode_token(token)
        user_id: int = payload.get("sub")
        if user_id is None:
            raise credentials_exception
    except JWTError:
        raise credentials_exception

    user = db.query(User).filter(User.id == user_id, User.is_active == True).first()
    if user is None:
        raise credentials_exception
    return user

def get_current_admin(current_user: User = Depends(get_current_user)) -> User:
    if current_user.role != "admin":
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Admin access required.",
        )
    return current_user
```

8 · Pydantic Schemas — User (app/schemas/user.py)

UserRegister enforces password strength (min 8 chars, 1 uppercase, 1 digit) via @field_validator. Email is validated by Pydantic's EmailStr type. UserOut never includes the hashed_password — it is excluded by design.

```
from __future__ import annotations
from datetime import datetime
from typing import Optional
from pydantic import BaseModel, EmailStr, field_validator
import re

class UserRegister(BaseModel):
    full_name: str
    email: EmailStr
    password: str
    role: Optional[str] = "user"

    @field_validator("password")
    @classmethod
    def password_strength(cls, v: str) -> str:
        if len(v) < 8:
            raise ValueError("Password must be at least 8 characters.")
        if not re.search(r"[A-Z]", v):
            raise ValueError("Password must contain at least one uppercase letter.")
        if not re.search(r"\d", v):
            raise ValueError("Password must contain at least one digit.")
        return v

    @field_validator("full_name")
    @classmethod
    def name_not_empty(cls, v: str) -> str:
        v = v.strip()
        if not v:
            raise ValueError("Full name cannot be empty.")
        return v

class UserLogin(BaseModel):
    email: EmailStr
    password: str

class UserOut(BaseModel):
    id: int
    full_name: str
    email: str
    role: str
    is_active: bool
    created_at: datetime

    class Config:
        from_attributes = True

class TokenResponse(BaseModel):
    access_token: str
    token_type: str = "bearer"
    user: UserOut

class UserUpdate(BaseModel):
    full_name: Optional[str] = None
    is_active: Optional[bool] = None
    role: Optional[str] = None
```

9 · Pydantic Schemas — Task (app/schemas/task.py)

TaskCreate validates status and priority against allowed enum values. TaskUpdate uses all Optional fields so clients can PATCH only changed fields. TaskListResponse wraps items with pagination metadata (total, page, per_page).

```
from __future__ import annotations
from datetime import datetime
from typing import Optional
from pydantic import BaseModel, field_validator

class TaskCreate(BaseModel):
    title: str
    description: Optional[str] = None
    status: Optional[str] = "todo"
    priority: Optional[str] = "medium"
    due_date: Optional[datetime] = None

    @field_validator("title")
    @classmethod
    def title_not_empty(cls, v: str) -> str:
        v = v.strip()
        if not v:
            raise ValueError("Title cannot be empty.")
        if len(v) > 200:
            raise ValueError("Title cannot exceed 200 characters.")
        return v

    @field_validator("status")
    @classmethod
    def valid_status(cls, v: str) -> str:
        if v not in ("todo", "in_progress", "done"):
            raise ValueError("status must be one of: todo, in_progress, done")
        return v

    @field_validator("priority")
    @classmethod
    def valid_priority(cls, v: str) -> str:
        if v not in ("low", "medium", "high"):
            raise ValueError("priority must be one of: low, medium, high")
        return v

class TaskUpdate(BaseModel):
    title: Optional[str] = None
    description: Optional[str] = None
    status: Optional[str] = None
    priority: Optional[str] = None
    due_date: Optional[datetime] = None

class TaskOut(BaseModel):
    id: int
    title: str
    description: Optional[str]
    status: str
    priority: str
    due_date: Optional[datetime]
    owner_id: int
    created_at: datetime
    updated_at: datetime

    class Config:
        from_attributes = True
```

```
class TaskListResponse(BaseModel):  
    total: int  
    page: int  
    per_page: int  
    items: list[TaskOut]
```

10 · CRUD — User (app/crud/user.py)

All database operations for users are isolated here. Emails are stored lowercase to prevent duplicate accounts with different casing. `update_user()` uses `setattr` to apply only the provided fields.

```
from sqlalchemy.orm import Session

from app.models import User
from app.schemas.user import UserRegister
from app.core.security import hash_password

def get_user_by_email(db: Session, email: str) -> User | None:
    return db.query(User).filter(User.email == email.lower()).first()

def get_user_by_id(db: Session, user_id: int) -> User | None:
    return db.query(User).filter(User.id == user_id).first()

def get_all_users(db: Session, skip: int = 0, limit: int = 50):
    return db.query(User).offset(skip).limit(limit).all()

def create_user(db: Session, data: UserRegister) -> User:
    user = User(
        full_name=data.full_name.strip(),
        email=data.email.lower(),
        hashed_password=hash_password(data.password),
        role=data.role or "user",
    )
    db.add(user)
    db.commit()
    db.refresh(user)
    return user

def update_user(db: Session, user: User, updates: dict) -> User:
    for key, val in updates.items():
        if val is not None:
            setattr(user, key, val)
    db.commit()
    db.refresh(user)
    return user

def delete_user(db: Session, user: User) -> None:
    db.delete(user)
    db.commit()
```

PYTHON

11 · CRUD — Task (app/crud/task.py)

`get_tasks()` always filters `is_deleted=False` (soft delete in action). Admin callers pass `owner_id=None` to get all tasks; regular users pass their own ID. `soft_delete_task()` sets `is_deleted=True` instead of issuing a DELETE statement.

```
from typing import Optional
from sqlalchemy.orm import Session

from app.models import Task, User
from app.schemas.task import TaskCreate, TaskUpdate

def get_tasks(
    db: Session,
    owner_id: Optional[int] = None,
    status: Optional[str] = None,
    priority: Optional[str] = None,
    skip: int = 0,
    limit: int = 20,
):
    q = db.query(Task).filter(Task.is_deleted == False)
    if owner_id is not None:
        q = q.filter(Task.owner_id == owner_id)
    if status:
        q = q.filter(Task.status == status)
    if priority:
        q = q.filter(Task.priority == priority)
    total = q.count()
    items = q.order_by(Task.created_at.desc()).offset(skip).limit(limit).all()
    return total, items

def get_task_by_id(db: Session, task_id: int) -> Task | None:
    return db.query(Task).filter(Task.id == task_id, Task.is_deleted == False).first()

def create_task(db: Session, data: TaskCreate, owner_id: int) -> Task:
    task = Task(
        title=data.title.strip(),
        description=data.description,
        status=data.status,
        priority=data.priority,
        due_date=data.due_date,
        owner_id=owner_id,
    )
    db.add(task)
    db.commit()
    db.refresh(task)
    return task

def update_task(db: Session, task: Task, data: TaskUpdate) -> Task:
    updates = data.model_dump(exclude_unset=True)
    for key, val in updates.items():
        setattr(task, key, val)
    db.commit()
    db.refresh(task)
    return task

def soft_delete_task(db: Session, task: Task) -> None:
    task.is_deleted = True
    db.commit()
```


12 · Auth Endpoints (app/api/v1/endpoints/auth.py)

Two public endpoints (no JWT required). /auth/register forces role='user' regardless of what the client sends — admins must be created by other admins via POST /users. /auth/login returns the token AND the user object so the frontend can display the user's name and role immediately.

```
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session

from app.database import get_db
from app.schemas.user import UserRegister, UserLogin, TokenResponse, UserOut
from app.crud.user import get_user_by_email, create_user
from app.core.security import verify_password, create_access_token

router = APIRouter(prefix="/auth", tags=["Authentication"])

@router.post("/register", response_model=UserOut, status_code=status.HTTP_201_CREATED)
def register(data: UserRegister, db: Session = Depends(get_db)):
    """Register a new user. Role defaults to 'user'."""
    if get_user_by_email(db, data.email):
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail="An account with this email already exists.",
        )
    # Only allow 'user' role via public registration
    data.role = "user"
    user = create_user(db, data)
    return user

@router.post("/login", response_model=TokenResponse)
def login(data: UserLogin, db: Session = Depends(get_db)):
    """Login and receive a JWT access token."""
    user = get_user_by_email(db, data.email)
    if not user or not verify_password(data.password, user.hashed_password):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid email or password.",
        )
    if not user.is_active:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Account is deactivated. Contact an administrator.",
        )
    token = create_access_token({"sub": user.id, "role": user.role})
    return TokenResponse(
        access_token=token,
        user=UserOut.model_validate(user),
    )
```

13 · User Endpoints (app/api/v1/endpoints/users.py)

GET /users/me — any authenticated user can fetch their own profile. GET /users — admin only, lists all users. PATCH /users/{id} — users can update their own full_name; only admins can change role or is_active. DELETE /users/{id} — admin only, hard delete.

```
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session

from app.database import get_db
from app.models import User
from app.schemas.user import UserOut, UserUpdate, UserRegister
from app.crud.user import (
    get_all_users, get_user_by_id, create_user,
    update_user, delete_user, get_user_by_email
)
from app.core.deps import get_current_user, get_current_admin
from app.core.security import hash_password

router = APIRouter(prefix="/users", tags=["Users"])

@router.get("/me", response_model=UserOut)
def get_me(current_user: User = Depends(get_current_user)):
    """Return the currently authenticated user's profile."""
    return current_user

@router.get("", response_model=list[UserOut])
def list_users(
    skip: int = 0,
    limit: int = 50,
    db: Session = Depends(get_db),
    _: User = Depends(get_current_admin),
):
    """[Admin only] List all registered users."""
    return get_all_users(db, skip=skip, limit=limit)

@router.get("/{user_id}", response_model=UserOut)
def get_user(
    user_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """Get a user by ID. Admins can view any user; users can only view themselves."""
    if current_user.role != "admin" and current_user.id != user_id:
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Access denied.")
    user = get_user_by_id(db, user_id)
    if not user:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="User not found.")
    return user

@router.post("", response_model=UserOut, status_code=status.HTTP_201_CREATED)
def create_user_admin(
    data: UserRegister,
    db: Session = Depends(get_db),
    _: User = Depends(get_current_admin),
):
    """[Admin only] Create a user with any role."""
    if get_user_by_email(db, data.email):
        raise HTTPException(status_code=status.HTTP_409_CONFLICT,
                            detail="Email already registered.")
    return create_user(db, data)
```

PYTHON

```

@router.patch("/{user_id}", response_model=UserOut)
def update_user_endpoint(
    user_id: int,
    data: UserUpdate,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """Update a user. Admins can update role/is_active; users can update own full_name."""
    if current_user.role != "admin" and current_user.id != user_id:
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Access denied.")
    user = get_user_by_id(db, user_id)
    if not user:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="User not found.")

    updates = data.model_dump(exclude_unset=True)
    if current_user.role != "admin":
        updates.pop("role", None)
        updates.pop("is_active", None)

    return update_user(db, user, updates)


@router.delete("/{user_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_user_endpoint(
    user_id: int,
    db: Session = Depends(get_db),
    _: User = Depends(get_current_admin),
):
    """[Admin only] Permanently delete a user."""
    user = get_user_by_id(db, user_id)
    if not user:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="User not found.")
    delete_user(db, user)

```

14 · Task Endpoints (app/api/v1/endpoints/tasks.py)

GET /tasks returns paginated results. Regular users see only their own tasks; admins see all. Supports ?status= and ?priority= query filters. PATCH uses model_dump(exclude_unset=True) so unset fields are not overwritten. DELETE is a soft-delete — the row stays in the DB with is_deleted=True.

```
from typing import Optional
from fastapi import APIRouter, Depends, HTTPException, Query, status
from sqlalchemy.orm import Session

from app.database import get_db
from app.models import User
from app.schemas.task import TaskCreate, TaskUpdate, TaskOut, TaskListResponse
from app.crud.task import (
    get_tasks, get_task_by_id, create_task, update_task, soft_delete_task
)
from app.core.deps import get_current_user, get_current_admin

router = APIRouter(prefix="/tasks", tags=["Tasks"])

@router.get("", response_model=TaskListResponse)
def list_tasks(
    status: Optional[str] = Query(None, description="Filter: todo | in_progress | done"),
    priority: Optional[str] = Query(None, description="Filter: low | medium | high"),
    page: int = Query(1, ge=1),
    per_page: int = Query(20, ge=1, le=100),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """
    List tasks. Regular users see only their own tasks.
    Admins see all tasks across all users.
    """
    owner_id = None if current_user.role == "admin" else current_user.id
    skip = (page - 1) * per_page
    total, items = get_tasks(db, owner_id=owner_id, status=status,
                             priority=priority, skip=skip, limit=per_page)
    return TaskListResponse(total=total, page=page, per_page=per_page,
                             items=[TaskOut.model_validate(t) for t in items])

@router.post("", response_model=TaskOut, status_code=status.HTTP_201_CREATED)
def create_task_endpoint(
    data: TaskCreate,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """Create a new task owned by the authenticated user."""
    task = create_task(db, data, owner_id=current_user.id)
    return TaskOut.model_validate(task)

@router.get("/{task_id}", response_model=TaskOut)
def get_task(
    task_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """Get a single task. Users can only view their own tasks; admins can view any."""
    task = get_task_by_id(db, task_id)
    if not task:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Task not found.")
    if current_user.role != "admin" and task.owner_id != current_user.id:
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Access denied.")
    return TaskOut.model_validate(task)
```

```

@router.patch("/{task_id}", response_model=TaskOut)
def update_task_endpoint(
    task_id: int,
    data: TaskUpdate,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """Update a task. Users can only update their own tasks."""
    task = get_task_by_id(db, task_id)
    if not task:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Task not found.")
    if current_user.role != "admin" and task.owner_id != current_user.id:
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Access denied.")
    return TaskOut.model_validate(update_task(db, task, data))

@router.delete("/{task_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_task_endpoint(
    task_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """Soft-delete a task. Users can only delete their own tasks."""
    task = get_task_by_id(db, task_id)
    if not task:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Task not found.")
    if current_user.role != "admin" and task.owner_id != current_user.id:
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Access denied.")
    soft_delete_task(db, task)

```

15 · API Router (app/api/v1/router.py)

Central router combines all three endpoint modules. Adding a new resource (e.g. /projects) only requires creating the endpoint file and adding one line here.

```
from fastapi import APIRouter
from app.api.v1.endpoints import auth, users, tasks

router = APIRouter()
router.include_router(auth.router)
router.include_router(users.router)
router.include_router(tasks.router)
```

PYTHON

16 · FastAPI Entry Point (app/main.py)

`lifespan()` calls `init_db()` at startup — creates all tables automatically. Global `RequestValidationError` handler converts Pydantic errors to a clean `{detail, errors}` format. CORS middleware allows all origins in development (restrict to your domain in production).

[illegible]

17 · Dockerfile & requirements.txt

Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc libpq-dev \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

DOCKERFILE

requirements.txt

Key packages and their purpose:

- fastapi 0.111 — async web framework with OpenAPI auto-docs
- uvicorn[standard] — ASGI server with HTTP/2 support
- sqlalchemy 2.0 — modern ORM with type-safe queries
- psycopg2-binary — PostgreSQL driver (binary = no libpq compile step)
- alembic — database migration tool (for future schema changes)
- pydantic[email] 2.7 — data validation with EmailStr support
- python-jose[cryptography] — JWT encode/decode with RS256/HS256
- passlib[bcrypt] — password hashing with bcrypt
- python-multipart — required by FastAPI for form data parsing

```
fastapi==0.111.0
uvicorn[standard]==0.29.0
sqlalchemy==2.0.30
psycopg2-binary==2.9.9
alembic==1.13.1
pydantic[email]==2.7.1
pydantic-settings==2.2.1
python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4
python-multipart==0.0.9
```

TXT

18 · docker-compose.yml

Four services: db (PostgreSQL 15), redis (Redis 7), backend (FastAPI), frontend (nginx serving static files). Health checks on db ensure the backend doesn't start until PostgreSQL is ready. nginx serves the frontend on port 3000 with zero configuration — just drop HTML/CSS/JS into /frontend/.

```
version: "3.9"

services:
  db:
    image: postgres:15-alpine
    restart: unless-stopped
    environment:
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: apppass
      POSTGRES_DB: taskdb
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U appuser -d taskdb"]
      interval: 5s
      timeout: 5s
      retries: 10

  redis:
    image: redis:7-alpine
    restart: unless-stopped
    ports:
      - "6379:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 10

  backend:
    build: ./backend
    restart: unless-stopped
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: postgresql://appuser:apppass@db:5432/taskdb
      REDIS_URL: redis://redis:6379/0
      SECRET_KEY: ${SECRET_KEY:-super-secret-key-change-in-production}
      ACCESS_TOKEN_EXPIRE_MINUTES: 60
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_healthy

  frontend:
    image: nginx:alpine
    restart: unless-stopped
    ports:
      - "3000:80"
    volumes:
      - ./frontend:/usr/share/nginx/html:ro
    depends_on:
      - backend

volumes:
  pgdata:
```

19 · Frontend — index.html

Single-page application. Two views: auth section (login/register tabs) and dashboard section (task creation form + task list). An edit modal overlays the dashboard for inline task editing. All transitions are handled by toggling `display:none` — no router needed.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>TaskManager - Primetrade.ai</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <div id="app">

    <!-- % AUTH SECTION % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % -->
    <div id="auth-section">
      <div class="auth-card">
        <div class="logo">&i TaskManager</div>
        <div class="tab-bar">
          <button class="tab active" onclick="showTab('login')">Login</button>
          <button class="tab" onclick="showTab('register')">Register</button>
        </div>

        <!-- Login Form -->
        <form id="login-form" onsubmit="handleLogin(event)">
          <label>Email</label>
          <input type="email" id="login-email" placeholder="you@example.com" required />
          <label>Password</label>
          <input type="password" id="login-password" placeholder="....." required />
          <button type="submit" class="btn-primary" id="login-btn">Login</button>
          <div id="login-msg" class="msg"></div>
        </form>

        <!-- Register Form -->
        <form id="register-form" style="display:none" onsubmit="handleRegister(event)">
          <label>Full Name</label>
          <input type="text" id="reg-name" placeholder="Vikas Sharma" required />
          <label>Email</label>
          <input type="email" id="reg-email" placeholder="you@example.com" required />
          <label>Password <span class="hint">(min 8 chars, 1 uppercase, 1 digit)</span></label>
          <input type="password" id="reg-password" placeholder="....." required />
          <button type="submit" class="btn-primary" id="reg-btn">Create Account</button>
          <div id="reg-msg" class="msg"></div>
        </form>
      </div>
    </div>

    <!-- % DASHBOARD SECTION % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % -->
    <div id="dashboard-section" style="display:none">
      <header>
        <div class="header-left">
          <span class="logo">&i TaskManager</span>
          <span id="user-badge"></span>
        </div>
        <div class="header-right">
          <span id="header-name"></span>
          <button class="btn-outline" onclick="logout()">Logout</button>
        </div>
      </header>

      <main>
        <!-- Create Task -->
        <section class="card">
          <h2>Create New Task</h2>

```

```
<form id="task-form" onsubmit="handleCreateTask(event)">
  <div class="form-row">
    <div class="form-group">
      <label>Title *</label>
      <input type="text" id="task-title" placeholder="Task title" required />
    </div>
    <div class="form-group">
      <label>Priority</label>
      <select id="task-priority">
        <option value="low">Low</option>
        <option value="medium" selected>Medium</option>
        <option value="high">High</option>
      </select>
    </div>
    <div class="form-group">
      <label>Status</label>
      <select id="task-status">
        <option value="todo" selected>To Do</option>
        <option value="in_progress">In Progress</option>
        <option value="done">Done</option>
      </select>
    </div>
    <div class="form-group">
      <label>Due Date</label>
      <input type="datetime-local" id="task-due" />
    </div>
  </div>
  <label>Description</label>
  <textarea id="task-desc" placeholder="Optional description..." rows="2"></textarea>
  <div class="form-actions">
    <button type="submit" class="btn-primary">Add Task</button>
    <div id="task-msg" class="msg"></div>
  </div>
</form>
</section>

<!-- Filters -->
<section class="card">
  <div class="filter-bar">
    <h2>My Tasks</h2>
    <div class="filters">
      <select id="filter-status" onchange="loadTasks()">
        <option value="">All Statuses</option>
        <option value="todo">To Do</option>
        <option value="in_progress">In Progress</option>
        <option value="done">Done</option>
      </select>
      <select id="filter-priority" onchange="loadTasks()">
        <option value="">All Priorities</option>
        <option value="low">Low</option>
        <option value="medium">Medium</option>
        <option value="high">High</option>
      </select>
      <button class="btn-outline" onclick="loadTasks()">!» Refresh</button>
    </div>
  </div>
  <div id="task-list"></div>
</section>
</main>
</div>

<!-- %% EDIT MODAL %%%%%%%%%%%%%%% -->
<div id="modal-overlay" style="display:none" onclick="closeModal()">
  <div class="modal" onclick="event.stopPropagation()">
    <h3>Edit Task</h3>
    <input type="hidden" id="edit-task-id" />
    <label>Title</label>
    <input type="text" id="edit-title" />
    <label>Description</label>
```

```
<textarea id="edit-desc" rows="2"></textarea>
<div class="form-row">
  <div class="form-group">
    <label>Status</label>
    <select id="edit-status">
      <option value="todo">To Do</option>
      <option value="in_progress">In Progress</option>
      <option value="done">Done</option>
    </select>
  </div>
  <div class="form-group">
    <label>Priority</label>
    <select id="edit-priority">
      <option value="low">Low</option>
      <option value="medium">Medium</option>
      <option value="high">High</option>
    </select>
  </div>
</div>
<div class="modal-actions">
  <button class="btn-primary" onclick="saveEdit()">Save Changes</button>
  <button class="btn-outline" onclick="closeModal()">Cancel</button>
</div>
<div id="edit-msg" class="msg"></div>
</div>
</div>
<script src="app.js"></script>
</body>
</html>
```

20 · Frontend — style.css

Responsive CSS-only design. Task status and priority badges are colour-coded (blue=todo, yellow=in_progress, green=done). The modal overlay uses position:fixed with a semi-transparent backdrop. No CSS framework or build step required.

[illegible]

[illegible]

[illegible]

21 · Frontend — app.js

Vanilla JS — no React, no build step. Token is stored in localStorage and attached to every API call via Authorization header. The api() helper detects 401 responses and auto-logs out. XSS protection via esc() escapes task content before inserting into innerHTML.

[illegible]

[illegible]

[illegible]

```
const res = await fetch(`${API}${path}`, {
  method, headers,
  body: body ? JSON.stringify(body) : undefined,
});
const isJson = res.headers.get("content-type")?.includes("application/json");
const data = isJson ? await res.json() : null;
if (res.status === 401) { logout(); return { ok: false, status: 401, data: { detail: "Session ..." } };
return { ok: res.ok, status: res.status, data };
} catch (err) {
  return { ok: false, status: 0, data: { detail: "Network error. Is the backend running?" } };
}
```

```
// %% Utils %%%%%%%%%%%%%%
```

```
function showMsg(el, text, type) {
  el.textContent = text;
  el.className = `msg ${type}`;
}
```

```
function formatError(data) {
  if (!data) return "Unknown error.";
  if (data.errors) return data.errors.map(e => e.message).join(", ");
  return data.detail || "Something went wrong.";
}
```

```
function esc(s) {
    return String(s).replace(/&/g, "&amp;").replace(/</g, "&lt;").replace(/>/g, "&gt;");
}
```

22 · README.md

MARKDOWN

TaskManager API – Backend Developer Intern Assignment

Scalable REST API with JWT Authentication, RBAC, and full CRUD for Tasks.
Built with ****FastAPI + PostgreSQL + Redis · Dockerised · Swagger UI included****.

Quick Start (single command)

```
```bash
git clone https://github.com/Vikas84214/task-manager.git
cd task-manager
cp .env.example .env # add your SECRET_KEY
docker compose up --build
```
```

| Service | URL |
|-------------|-----------------------------|
| API | http://localhost:8000 |
| Swagger UI | http://localhost:8000/docs |
| ReDoc | http://localhost:8000/redoc |
| Frontend UI | http://localhost:3000 |

Example curl Commands

Register

```
```bash
curl -X POST http://localhost:8000/api/v1/auth/register \
 -H "Content-Type: application/json" \
 -d '{"full_name":"Vikas","email":"vikas2227031583@gmail.com","password":"Secret123"}'
```
```

Login

```
```bash
curl -X POST http://localhost:8000/api/v1/auth/login \
 -H "Content-Type: application/json" \
 -d '{"email":"vikas2227031583@gmail.com","password":"Secret123"}'
!' {"access_token": "<JWT>", "token_type": "bearer", "user": {...}}
```
```

Create Task

```
```bash
TOKEN="<paste JWT here>"
curl -X POST http://localhost:8000/api/v1/tasks \
 -H "Authorization: Bearer $TOKEN" \
 -H "Content-Type: application/json" \
 -d '{"title":"Build the API","priority":"high","status":"in_progress"}'
```
```

List Tasks

```
```bash
curl -H "Authorization: Bearer $TOKEN" \
 "http://localhost:8000/api/v1/tasks?status=in_progress&page=1&per_page=10"
```
```

Update Task

```
```bash
curl -X PATCH http://localhost:8000/api/v1/tasks/1 \
 -H "Authorization: Bearer $TOKEN" \
 -H "Content-Type: application/json" \
 -d '{"status":"done"}'
```
```

```
### Delete Task
```bash
curl -X DELETE http://localhost:8000/api/v1/tasks/1 \
 -H "Authorization: Bearer $TOKEN"
```
```

Scalability Note

| Concern | Current | At Scale |
|---------------|-----------------|--|
| API instances | 1 (Docker) | N replicas behind nginx / AWS ALB |
| Database | PostgreSQL | RDS Multi-AZ + read replicas |
| Caching | None | Redis (token blacklist, query cache) |
| Auth tokens | Stateless JWT | Add Redis token blacklist for instant revoke |
| Logging | Uvicorn stdout | Structured JSON !' Loki / CloudWatch |
| Deployment | Docker Compose | Kubernetes + Helm, HPA on CPU/RPS |
| DB migrations | auto create_all | Alembic versioned migrations |

23 · Example curl Commands & API Response Shapes

Register a new user

```
curl -X POST http://localhost:8000/api/v1/auth/register \
-H "Content-Type: application/json" \
-d '{
  "full_name": "Vikas",
  "email": "vikas2227031583@gmail.com",
  "password": "Secret123"
}'

# 201 Created
{
  "id": 1,
  "full_name": "Vikas",
  "email": "vikas2227031583@gmail.com",
  "role": "user",
  "is_active": true,
  "created_at": "2024-06-10T10:00:00"
}

# 409 if email already exists
{ "detail": "An account with this email already exists." }

# 422 if password too weak
{ "detail": "Validation failed.", "errors": [ { "field": "body.password", "message": "Password must..." }
```

Login

```
curl -X POST http://localhost:8000/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"email": "vikas2227031583@gmail.com", "password": "Secret123"}'

# 200 OK
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer",
  "user": { "id": 1, "full_name": "Vikas", "email": "...", "role": "user" }
}

# 401 if wrong credentials
{ "detail": "Invalid email or password." }
```

Create a Task (authenticated)

```
TOKEN="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

curl -X POST http://localhost:8000/api/v1/tasks \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "title": "Build the API",
  "description": "FastAPI + PostgreSQL + JWT",
  "priority": "high",
  "status": "in_progress",
  "due_date": "2024-12-31T23:59:00"
}'

# 201 Created
{
  "id": 1,
  "title": "Build the API",
```

```
"description": "FastAPI + PostgreSQL + JWT",
"status": "in_progress",
"priority": "high",
"due_date": "2024-12-31T23:59:00",
"owner_id": 1,
"created_at": "2024-06-10T10:05:00",
"updated_at": "2024-06-10T10:05:00"
}
```

List, Update, Delete Tasks

```
# List tasks (paginated, filterable)
curl -H "Authorization: Bearer $TOKEN" \
  "http://localhost:8000/api/v1/tasks?status=in_progress&priority=high&page=1&per_page=10"

# Response:
{ "total": 1, "page": 1, "per_page": 10,
  "items": [{ "id": 1, "title": "Build the API", "status": "in_progress", ... }] }

# Update task (PATCH – only send changed fields)
curl -X PATCH http://localhost:8000/api/v1/tasks/1 \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"status": "done"}'

# Delete task (soft delete)
curl -X DELETE http://localhost:8000/api/v1/tasks/1 \
  -H "Authorization: Bearer $TOKEN"
# 204 No Content

# Get my profile
curl -H "Authorization: Bearer $TOKEN" http://localhost:8000/api/v1/users/me
```

Admin — List all users

```
# Must be logged in as admin role
ADMIN_TOKEN="..."

curl -H "Authorization: Bearer $ADMIN_TOKEN" \
  http://localhost:8000/api/v1/users

# 403 if not admin
{ "detail": "Admin access required." }

# Create admin user (admin only)
curl -X POST http://localhost:8000/api/v1/users \
  -H "Authorization: Bearer $ADMIN_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "full_name": "Admin User",
    "email": "admin@example.com",
    "password": "Admin123",
    "role": "admin"
  }'
```

24 · Role-Based Access Control Matrix

| Endpoint | Method | user role | admin role |
|----------------------------|--------|---------------|---------------|
| POST /api/v1/auth/register | POST | ' Public | ' Public |
| POST /api/v1/auth/login | POST | ' Public | ' Public |
| GET /api/v1/users/me | GET | ' Own profile | ' Own profile |
| GET /api/v1/users | GET | 'L 403 | ' All users |
| GET /api/v1/users/{id} | GET | ' Own only | ' Any user |
| POST /api/v1/users | POST | 'L 403 | ' Any role |
| PATCH /api/v1/users/{id} | PATCH | ' Own name | ' Full update |
| DELETE /api/v1/users/{id} | DELETE | 'L 403 | ' Any user |
| GET /api/v1/tasks | GET | ' Own tasks | ' All tasks |
| POST /api/v1/tasks | POST | ' Create | ' Create |
| GET /api/v1/tasks/{id} | GET | ' Own only | ' Any task |
| PATCH /api/v1/tasks/{id} | PATCH | ' Own only | ' Any task |
| DELETE /api/v1/tasks/{id} | DELETE | ' Own only | ' Any task |

25 · Scalability & Production Readiness Notes

This section is the 'short scalability note' required by the assignment deliverables.

Current Strengths

- Stateless JWT — no server-side session storage; any API replica can serve any request
- Connection pooling (SQLAlchemy pool_size=10, max_overflow=20) — handles concurrent requests efficiently
- Soft delete — data is preserved for audit trails and potential recovery
- API versioning (/api/v1/) — breaking changes go in /api/v2/ without disrupting existing clients
- Layered architecture (routes != crud != db) — each layer is independently testable
- Pydantic v2 validation — all inputs sanitised before reaching the database

Path to Production Scale

| Concern | Now | At Scale Solution |
|------------------|------------------------|---|
| API instances | 1 Docker container | N replicas behind nginx or AWS ALB (stateless JWT = trivially horizontal) |
| Database | PostgreSQL single node | RDS Multi-AZ primary + 1-2 read replicas; GET endpoints route to replicas |
| Caching | No cache | Redis: cache GET /tasks results (TTL=30s); blacklist revoked JWTs |
| Token revocation | Not supported | Redis set of invalidated JTI claims; check on every request |
| Logging | uvicorn stdout | Structured JSON logs != Loki / CloudWatch; correlate by request-id header |
| Rate limiting | None | nginx limit_req or API Gateway throttling; Redis sliding-window counter |
| Background tasks | None | Celery + Redis for email notifications, report generation |
| Migrations | auto create_all | Alembic versioned migrations — safe schema changes with rollback |
| Secrets | .env file | AWS Secrets Manager / Vault; rotate SECRET_KEY without downtime |
| Deployment | Docker Compose | Kubernetes + Helm chart; HPA on CPU/RPS; KEDA for background workers |

Microservices Evolution Path

The current monolith is structured to split cleanly into microservices when needed:

- 1. Auth Service — handles registration, login, token issuance. Other services validate tokens locally.
- 2. User Service — profile management, RBAC. Uses an internal gRPC or REST API.
- 3. Task Service — CRUD for tasks. Publishes TaskCreated / TaskCompleted events to a message bus (Kafka/RabbitMQ).
- 4. Notification Service — subscribes to task events, sends email/SMS/push alerts.
- 5. API Gateway — single entry point for all clients; handles routing, rate limiting, auth token forwarding.